

In the name of GOD.



Sharif University of Technology

Stochastic Processes

Fall 2025

Hamid R. Rabiee

Final Project: The Black Box Trader

Deadline: 1404/11/15

Introduction

This project places you in a realistic quantitative finance scenario. You are provided with a dataset generated by a complex, proprietary Market Simulation Engine.

Your goal is twofold:

1. **Reverse Engineer the Math:** Analyze the training data to deduce the underlying stochastic processes driving the prices.
2. **Beat the Market:** Develop a trading algorithm based on your findings. This algorithm will be run against a hidden Test Set (same underlying physics, different random seed/parameters) on Evaluation Day.

Market Intelligence (Hints)

The simulation engine is not a simple Random Walk. It combines several stochastic mechanisms. Use the following qualitative intelligence reports to guide your mathematical modeling:

- **Market Phases:** The market is not static. It shifts abruptly between distinct behavioral modes (e.g., volatile vs. stable). You need to detect the current phase.
- **Periodicity:** The data exhibits repeating, cyclical patterns over time overlaid on the stochastic noise.
- **Supply and Demand:** Prices are driven by the fundamental imbalance between supply and demand in the order flow.

- **Elasticity:** Prices exhibit mean-reverting tendencies, often pulling back to a central value after significant deviations.
- **Leading Indicators:** Certain non-tradeable indices in the dataset display a predictive correlation with the tradeable assets, often anticipating movements.

Platform and Data Access

All datasets and submissions are managed through the project portal:

<http://185.231.180.74:3000>

- **Training Set:** Available for download immediately. Use this to reverse-engineer the market structure.
- **Evaluation Set:** Available for testing your code locally.
- **Test Set:** Hidden. This will only be processed on the server on the deadline.

Deliverables

Part 1: The Technical Report (PDF)

You must submit a report explaining how you modeled the market.

- Explain the mathematical structure you discovered behind the observed market dynamics and patterns.
- detail your parameter estimation methods.
- Justify your trading strategy based on these theoretical findings.

Part 2: The Trading Script (Python)

You must upload a Python script to the server. Your script must implement the TradingStrategy class.

The server will feed market data to your script tick-by-tick.

Interface Contract:

```

from typing import List, Dict

from trading_data_structures import (
    COMPANY_SYMBOLS,
    EXTERNAL_SYMBOLS,
    PortfolioState,
    Order,
)

class TradingStrategy:
    def __init__(self):
        """Called once at the start of the simulation."""
        pass

    def on_tick(self, assets_map: Dict[str, "AssetData"], portfolio: PortfolioState,) -> List[Order]:
        """
        Processes a single market tick.
        Return a list of orders for the current tick.
        """
        orders: List[Order] = []

        return orders

```

Rate-Limit & Quotas

- File size: max 1 MiB, only .py.
- Execution timeout: 5 min.
- Sandbox resources: 2 GiB RAM, no network, no extra package, read-only bind mount.

Some Additional Notes

- Because internet connectivity is unreliable in the country, the project is designed so that no external Python packages need to be installed; everything must run with the standard library that comes with Python 3.10.
- After downloading the dataset you can use the dataset preview page in website to visualize the data streams and get an intuitive feel for the dynamics before any modeling work begins.
- The competition follows a three-stage pipeline – train, eval, and test – and all three stages are generated by the same underlying simulation engine, but the engine's internal parameters differ for each stage.
- In the train stage you are expected to reverse-engineer the stochastic processes that drive the price series and build a mathematical model of the market.
- The test stage allows only a single submission; the quality of that one answer determines the final score.
- Success therefore depends on a robust, parameter-agnostic modeling framework that can be fitted quickly and reliably on unseen data.
- The final deliverables are a technical PDF report explaining the discovered mathematics and a Python script implementing the TradingStrategy class.

Grading Policy (Total: 100 Points)

Your grade is determined by your understanding of the underlying physics and your ability to exploit them.

- **50 Points: The Report**

Did you correctly identify the underlying mechanics? We are looking for mathematical evidence that you "cracked" the simulation code's logic, not just lucky guesses. You must justify your findings mathematically using concepts within the scope of this course. Do not use methods beyond the scope of the course.

- **50 Points: Live Performance**

Your script will be run against the Test Set on the server.

- Passing: Positive returns.
- Full Points: Outperforming the Benchmark Strategy.

- **Bonus: Top Performers**

The top submissions, ranked by total return on the hidden Test Set, will receive a bonus mark.